Tesis Técnica de Ingeniería de Software: Proyecto FixFinder

FixFinder representa una solución avanzada de software distribuido. Este documento detalla la ingeniería subyacente, el diseño de concurrencia y la arquitectura de comunicaciones que garantizan la escalabilidad y robustez del sistema en entornos de alta demanda.

---

1. Arquitectura Unificada de Concurrencia (Gestión de Hilos)

La gestión de hilos es el sistema circulatorio del proyecto. FixFinder utiliza modelos de concurrencia diferenciados para optimizar el rendimiento de cada plataforma.

A. Backend: Modelo Dispatcher-Worker y Control de Flujo

El servidor Java opera de forma puramente multihilo para atender peticiones concurrentes sin bloqueos mutuos.

- **Hilo Dispatcher (El Orquestador):** Este hilo vive en un bucle infinito bloqueado en la instrucción `serverSocket.accept()`. Su única responsabilidad es detectar una petición de conexión entrante y "despacharla" delegándola a un hilo trabajador. Es vital que sea ligero; no realiza ninguna operación de entrada/salida (I/O) ni lógica de negocio.
- **Hilos Workers (Ejecución Síncrona Masiva):** Por cada cliente conectado, se instancia un objeto `GestorConexion` que corre en su propio **Thread físico**. Este hilo asume la responsabilidad total de la conexión:
  - **Lectura Binaria:** Extrae el mensaje de la red.
  - **Lógica de Negocio:** Llama a los servicios correspondientes.
  - **Persistencia SQL:** Es este mismo hilo el que ejecuta las sentencias SQL contra MySQL. No se delega a otro hilo para evitar cambios de contexto (context switching) innecesarios, garantizando que el usuario reciba la respuesta solo cuando la base de datos ha confirmado el guardado.
- **Control de Saturación (Semaphore):** Implementamos un semáforo de concurrencia de 10 permisos. Esto actúa como un cortafuegos: si 10 hilos están procesando peticiones pesadas, el Dispatcher bloquea nuevas conexiones hasta que un Worker termine, protegiendo así la integridad de la memoria RAM y el pool de conexiones.

B. Dashboard Desktop: El Triángulo de Red (Lector Avaro y Desacoplo)

En JavaFX, leer directamente del socket desde la interfaz de usuario es inviable. Hemos diseñado una arquitectura de tres hilos:

1. **Hilo de UI (JavaFX Main):** Gestiona el renderizado a 60 FPS y los eventos del ratón. No toca la red para evitar que la ventana se cuelgue.
2. **Hilo Lector Avaro (TCP Flow Scavenger):** Este hilo es crítico. TCP utiliza una "ventana de recepción". Si la aplicación no lee los datos rápido, esa ventana se llena y el Sistema Operativo le dice al servidor: "No me envíes más datos, estoy lleno". El Lector Avaro está en un bucle cerrado leyendo bytes a velocidad máxima y "vaciando el tubo" hacia una cola interna (`LinkedBlockingQueue`).

3. **Hilo Procesador (Consumer):** Este hilo está escuchando la cola interna. En cuanto llega un JSON, lo procesa. Si los datos implican un cambio visual (como actualizar una tabla), invoca a `Platform.runLater()`, que es el mecanismo seguro para pedirle al hilo de UI que pinte los nuevos datos.

C. App Móvil: El Modelo de Event Loop (Dart)

A diferencia de Java, Flutter (Dart) es monohilo por defecto pero altamente asíncrono.

- **Event Loop:** Funciona como una cola infinita de tareas. Cuando la App hace una petición de red, no se bloquea; delega la espera al Sistema Operativo y sigue pintando la interfaz.
- **Asincronía Real:** Cuando los datos llegan al Socket, el sistema lanza un evento a la cola y el hilo principal procesa el mensaje solo cuando tiene un hueco libre entre fotogramas de la interfaz. Esto da la sensación de fluidez total aunque solo usemos un procesador.

---

2. El Viaje del Dato: Capas del Backend y Responsabilidades

Para entender cómo una acción en el móvil se convierte en una fila en la base de datos, debemos desglosar la "Cadena de Responsabilidades":

1. **Capa 4: TCP Socket (Entrada):** Los bytes brutos llegan al Servidor. El `GestorConexion` reconstruye el mensaje basándose en la cabecera de 4 bytes (Big-Endian).
2. **Capa de Enrutamiento: Procesadores:** El sistema identifica la clave `"accion"` (ej: `CREAR_PRESUPUESTO`) y delega el mensaje al `ProcesadorPresupuestos`. Esta capa es la frontera de seguridad: parsea el JSON y valida que los campos técnicos existan.
3. **Capa de Negocio: Services:** El procesador llama al `PresupuestoServiceImpl`. Aquí es donde reside la inteligencia del proyecto. Se ejecutan las comprobaciones lógicas: "¿Existe el trabajo?", "¿Es el monto positivo?", "¿Puede este usuario enviar ofertas?".
4. **Capa de Persistencia: DAOs:** El Servicio llama al `PresupuestoDAOImpl`. Este componente es el especialista en SQL. Traduce los objetos de Java (`Presupuesto`) a sentencias `INSERT INTO` o `UPDATE`.
5. **Capa de Datos: MySQL:** Finalmente, el driver JDBC envía la sentencia al motor de base de datos. Una vez confirmada, el resultado recorre el camino inverso hasta la App.

---

3. Lógica de Multipresupuesto 1:N: Atomicidad y Transaccionalidad

Este es el mayor desafío técnico reciente. Hemos pasado de un presupuesto por trabajo a un sistema de subasta libre.

- **Atomicidad del Aceptado:** Al aceptar un presupuesto, el servidor debe garantizar que el estado sea consistente. No puede ser que un presupuesto se acepte y los otros sigan pendientes.
- **Estrategia de Barrido de Competidores:** El método `aceptarPresupuesto` realiza un proceso coordinado:
    1. Marca el presupuesto elegido como `ACEPTADO` (Estado 1 en el Enum).
    2. Busca mediante una consulta SQL a todos los rivales del mismo trabajo e ID diferente y los cambia masivamente a `RECHAZADO`.
    3. Actualiza el estado de la incidencia a `ACEPTADO`.

- **Trazabilidad mediante Inyección Estructural:** Para que el operario sepa qué se ha acordado, hemos programado un sistema de inyección de texto. El servidor busca marcadores específicos (💰 `GERENTE:`) dentro de la descripción del trabajo y, mediante manipulación de cadenas de alta precisión, inserta las notas técnicas de la oferta ganadora sin borrar el mensaje inicial del cliente.

---

4. Persistencia Aislada mediante ThreadLocal

En un servidor multihilo, el mayor peligro es que dos hilos compartan la misma conexión a la DB.

- **Aislamiento Galvánico:** Usamos `ThreadLocal<Connection>`. Es un patrón de diseño donde cada hilo Worker tiene su propia variable privada de conexión. Es físicamente imposible que una transacción del Usuario A se mezcle con la del Usuario B.
- **Gestión de Fugas (Memory Leaks):** El servidor implementa un bloque `finally` obligatorio en la lectura de cada mensaje. Pase lo que pase (incluso si hay un error fatal), el sistema garantiza que la conexión del hilo se cierra y se elimina del `ThreadLocal`, dejando el servidor limpio para la siguiente petición.

---

5. El Protocolo Binario (Arquitectura de Cabecera Big-Endian)

Para lograr una comunicación de nivel industrial, hemos diseñado un protocolo de capa de aplicación propio que resuelve la fragmentación de paquetes TCP mediante una cabecera de longitud fija.

A. El Integer de 32 bits (Base 256)

En lugar de enviar la longitud en formato texto (ASCII), lo enviamos como un flujo binario de 4 bytes. Esto nos permite un **empaquetamiento masivo**:

- **Eficiencia Matemática:** Mientras que 4 bytes en formato texto solo podrían representar longitudes hasta 9,999 (10KB), 4 bytes en binario (Base 256) pueden direccionar hasta **4,29 Gigabytes**.
- **Determinismo:** El receptor (ya sea Java o Flutter) sabe que debe leer **exactamente los primeros 4 bytes** para conocer el tamaño del mensaje. Esto elimina la ambigüedad y el "clipping" de mensajes en la red.

B. Cálculo de Pesos y Endianness

Implementamos el estándar **Big-Endian** (Most Significant Byte first), donde el orden de los bytes sigue una jerarquía de potencias de 256: $$Longitud = (b0 \cdot 256^3) + (b1 \cdot 256^2) + (b2 \cdot 256^1) + (b3 \cdot 256^0)$$ Esta arquitectura garantiza que, independientemente de si la CPU del cliente es Little-Endian (Intel) o Big-Endian (algunos ARM), el protocolo de red sea el "árbitro" que unifique el valor numérico.

C. Reconstrucción Asíncrona y Buffer de Seguridad

Tanto en Java como en Dart, hemos programado un buffer acumulativo de seguridad:

1. **Vaciado del Socket:** El sistema lee fragmentos de red.
2. **Acumulación:** Si un mensaje llega troceado (ej: JSON grande de incidencias), los bytes se guardan en un buffer temporal.

3. **Disparo (Firing):** Solo cuando la longitud acumulada coincide exactamente con el valor calculado en la cabecera, se dispara el parseo JSON. Esto evita errores de sintaxis por mensajes incompletos.

---

6. Conclusión de Ingeniería

FixFinder demuestra un dominio total de las capas inferiores del desarrollo de software:

1. **Gestión de Red:** Control manual de sockets y buffers.
2. **Gestión de Concurrencia:** Arquitecturas asíncronas de 3 hilos y dispatchers.
3. **Gestión de Datos:** Transaccionalidad atómica y aislamiento de hilos mediante patrones de diseño avanzados.